Adding a product (with color-preview masks)

BP17 gave the storefront live color preview: the customer picks colors on the PDP and the product photo recolors in real time. It works by compositing the photo with one **alpha mask per recipe slot** (`product_image_masks` table, files under `uploads/store/masks/`). This doc is the ongoing process — what to do so every new product gets masks as part of the normal flow.

Photo rule (matters more than anything below)

Shoot base photos of a print in WHITE / light-neutral filament. The storefront tints with a multiply blend — it can only darken. A white base renders every color faithfully; a colored base (e.g. the blue can cooler) goes muddy when tinted. The Phase 1 spike (`wiz3d-prints/scripts/color-spike/DECISION.md`) has the visual evidence.

The upload pipeline still does its usual thing (rembg background removal, crop, dark-backdrop composite) — nothing new to do there.

Single-color products (recipe has exactly 1 slot)

Nothing to do. On photo upload the pipeline saves the silhouette mask automatically (green `MASK: AUTO` pill on the image tile).

If a product's images pre-date this (gray `NO MASK` pill):

- Per image: **Products** → **edit product** → **hover the image** → **Generate Mask**, or
- Catalog-wide: **Products index** → **Generate Masks** (top-right) — masks every unmasked image of every single-color product, shows progress + failures.

A red `MASK FAILED` pill means rembg produced a degenerate mask (reason on hover) — fix the photo or retry.

Multi-color products (recipe has 2+ slots)

Masks are prepared by hand, one PNG per recipe slot, then uploaded in the admin (**hover the image** → **Edit Masks**).

Prep (Photoshop / GIMP / Procreate):

1. Open the PROCESSED product photo (the one the storefront shows — download it from the product page image tile).

2. One layer per recipe slot. On each layer, paint over the area that slot's filament covers. **Opaque = this slot's color applies here; transparent = leave alone.** Layer color doesn't matter (the server normalizes) — only the transparency does.
3. Export each layer as its own PNG **with transparency**, same canvas size as the photo (other sizes work — they're scaled — but same-size is foolproof). Name them `slot-0.png`, `slot-1.png`, … matching the recipe order shown in the Edit Masks dialog (slot 1 in the UI = the recipe's first color).
4. Upload each PNG on its slot row in **Edit Masks**. Bad exports are rejected with the reason (no alpha channel / empty / covers everything).

Verify before you're done — in the same dialog:

- **Slot Overlay** — each mask tinted a distinct color (key shown per row). Wrong slot assignment or sloppy coverage is instantly visible.
- **Test Colors** — the real storefront composite. Try wild combos; if it looks right here it looks right on the PDP.

The image tile shows a yellow `MASK m/N` pill until every slot is covered — the storefront only recolors an image whose masks cover **all** slots (otherwise it falls back to the plain photo, so partial coverage is safe but invisible to customers).

Replacing/removing a mask archives the old file for 30 days (`uploads/store/masks/archive/` on CT 114) — an accidental upload is recoverable.

Kill switch (storefront)

`ENABLE_DYNAMIC_COLOR_PREVIEW=false` in wiz3d-prints' `.env` on CT 308 (`/home/shad/docker/wiz3d_prints/.env`) + `docker compose up -d` reverts the whole storefront to plain photos — runtime flag, no rebuild. Unset = on.

Where things live

Thing	Where
Mask rows	`product_image_masks` (migration 043) — `(image_id, slot_index)` unique
Mask files	CT 114 `uploads/store/masks/` (30-day archive in `masks/archive/`)
Generation	rembg sidecar `wiz3dtools-rembg` :7000 + `mask-generator.service.ts`
Store API	`GET /api/store/products` → `images[].masks[{slotIndex,url}]`
Storefront renderer	wiz3d-prints `components/shop/ProductRenderer.tsx` + `lib/compositor.ts` (copy of `packages/frontend/src/lib/compositor.ts` here — keep in sync)